AWS Architecture Diagram

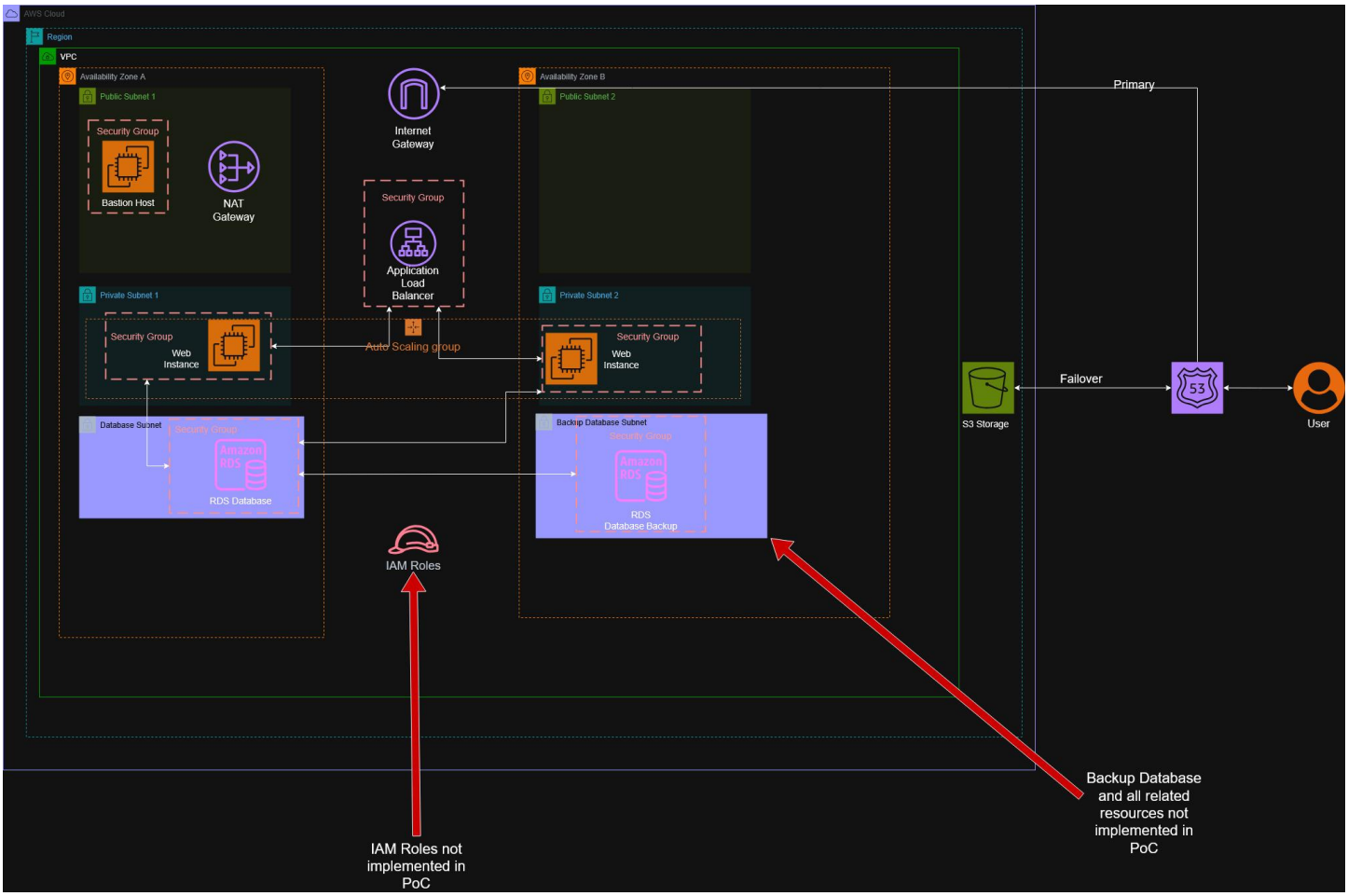


Figure 1 AWS Architecture Diagram

The above figure illustrates the AWS architecture solution based on the WinLocal Giveaways scenario. The solution implements a separation of the web, application and database layers hosted within a virtual private cloud with a public facing web tier, a private application tier and finally an isolated database tier, all spanning across two availability zones. This architecture design directly addresses WinLocal's issues and requirements of high availability, scalability, security and sustainability by ensuring there is no longer a single point of failure and that each layer is isolated and protected through segmentation of the network and security groups. The end user first meets the Route 53 DNS entry point to the architecture, being directed either to the application load balancer, which or to the backup site in S3 storage if the webpage is down. The ALB distributes traffic across EC2 web instances which are managed by an auto scaling group in the private subnets. A NAT gateway in the public subnet allows the private web instances outbound access to the internet for the purpose of patching or updating, without the private instances being exposed to the public internet. The databases are located within their own isolated subnets and security groups, hosting an Amazon RDS MySQL instance that is only accessible via the application layer, ensuring customer data is isolated and protected from the public internet with a replica backup database provisioned to ensure the database is readily available and accessible even in the event of a failure. A bastion host provisioned in the public subnet provides the only administrative access route to the private web instances, strictly through SSH with an encrypted key pairing setup, removing the need for any public exposure of the private servers. The IAM roles, backup database and Route 53 services are not included in the proof of concept and are theoretical additions purely for the logical diagram and potential improvements in the proposed architecture solution.

Proof Of Concept

Network Design

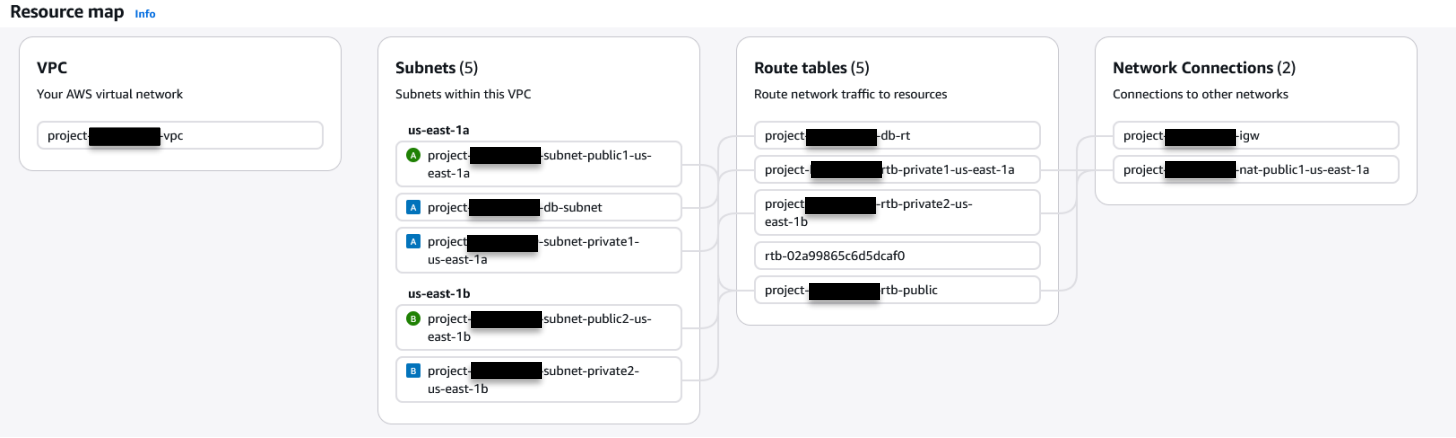


Figure 2 VPC Resource Map

The above VPC resource map was configured with a CIDR of 10.0.0.0/16, providing a network environment for all WinLocal specific resources. The network is comprised of two public subnets and two private subnets, spanning across two availability zones, and an isolated database subnet. An internet gateway was attached to the VPC to allow public internet traffic to access the WinLocal webpage, while a NAT gateway was created in the public subnet to allow the private web instances to access the internet for updates and patches, without directly exposing them to public access. Separate route tables were implemented for each subnet type, the public route table routes traffic through the internet gateway, the private route table directs outbound traffic through the NAT gateway and the database route table doesn't contain an outbound route, ensuring complete isolation and protection of customer data from public exposure. This network segmentation design solves WinLocal's flat, unsegmented network architecture by enforcing separation between all three layers and preventing a breach at any one layer from moving laterally to any other layer.

Bastion Host

A bastion host EC2 instance was deployed in the public subnet of availability zone A. This bastion host serves as the only entry route into the private WinLocal network for purposes of administration. The bastion's security group was set up to allow inbound SSH access on port 22 exclusively from a single, trusted IP address, preventing any unauthorised access through the use of an encrypted key pair.

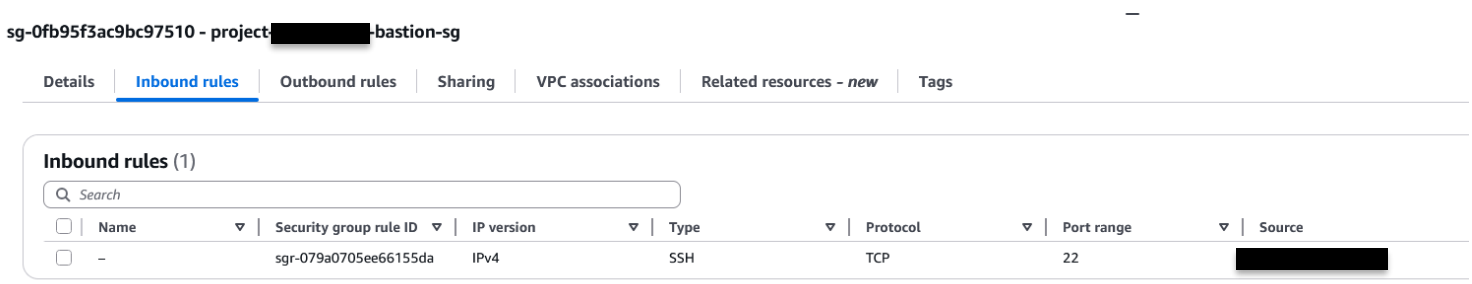


Figure 3 Security Group Allowing Restricted SSH Access to Bastion Host from Single, Trusted IP

Through this bastion host administrative users can SSH into the private web servers either using agent forwarding or by uploading the private key onto the bastion host. This excises the need for the web servers to have a public IP address, keeping them unreachable directly from the internet. Without this bastion host, admin access to the web servers would require them either having a public IP address, exposing them to the internet, or might require access through less secure protocols. Having a single entry point also makes it significantly easier to monitor and audit access and admin activity, aiding non-repudiation.

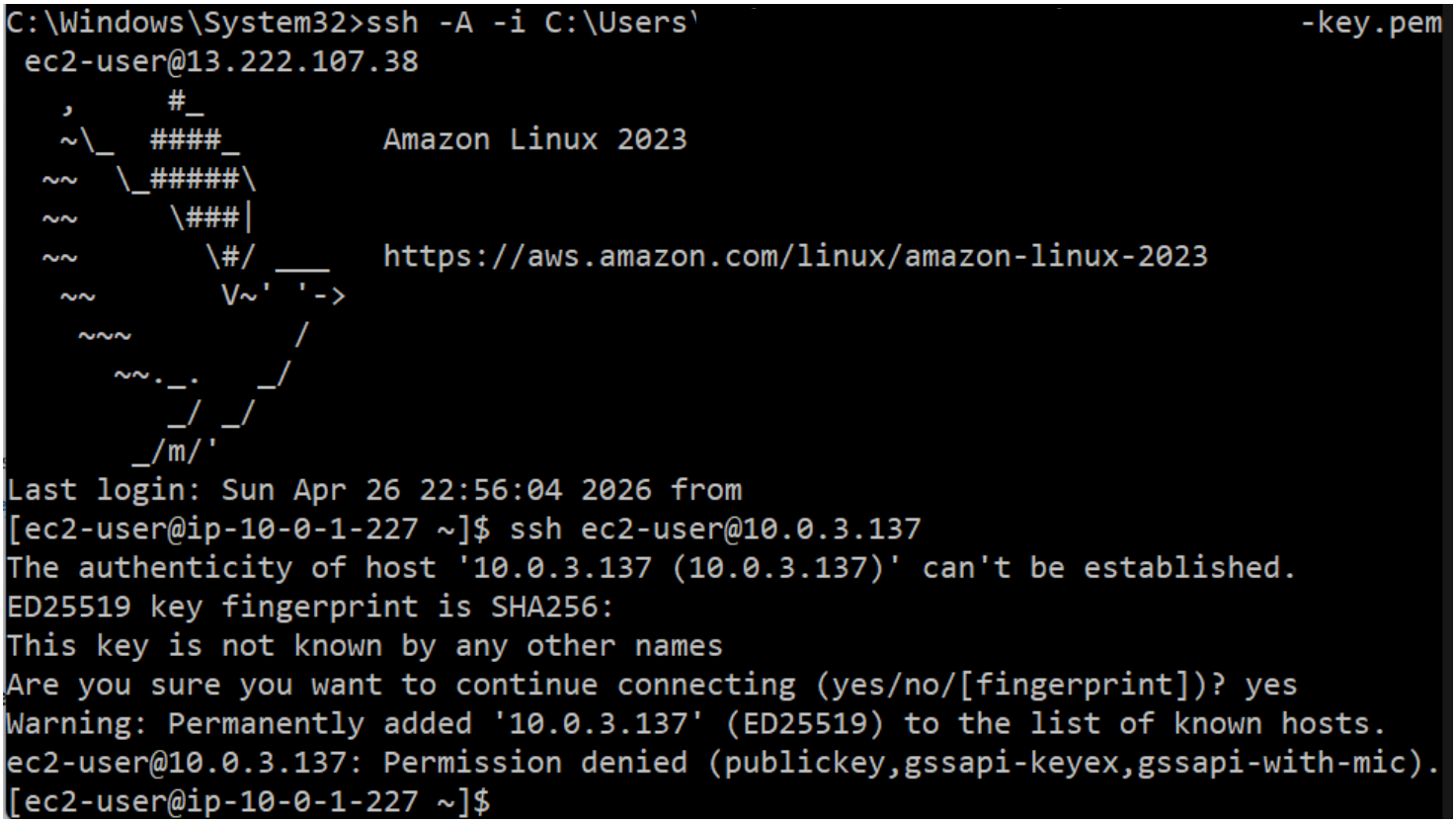


Figure 4 Private Web Instance SSH Access via Bastion Host

Amazon Machine Image (AMI)

An Amazon machine image was created from a running EC2 web instance once it had been configured with the WinLocal web application. Amazon machine image's capture an instance in its running state and can then be used to launch instances with the same configuration, providing a template from which identical instances can be launched when necessary. The AMI created from the WinLocal web app instance was used for the auto scaling group launch template, so that every instance provisioned by the auto scaling group would be pre-configured with the web app and ready to handle internet traffic without any manual intervention, as was previously the case at WinLocal. AMIs guarantee that every instance is identical, removing the risk of inconsistencies when new instances are launched.

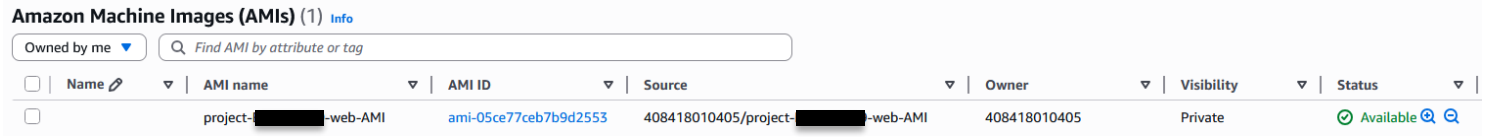


Figure 5 AMI Created from Running Web Instance

Amazon RDS Deployment

An RDS MySQL database instance was deployed within the isolated database subnet of availability zone A, with the goal of complete isolation and protection from public access via a dedicated security group that only allows inbound connection on port 3306 for MySQL query access from the private web servers. The web app was configured to connect to the RDS endpoint with successful connectivity being confirmed by the appearance of the Address Book in the browser.

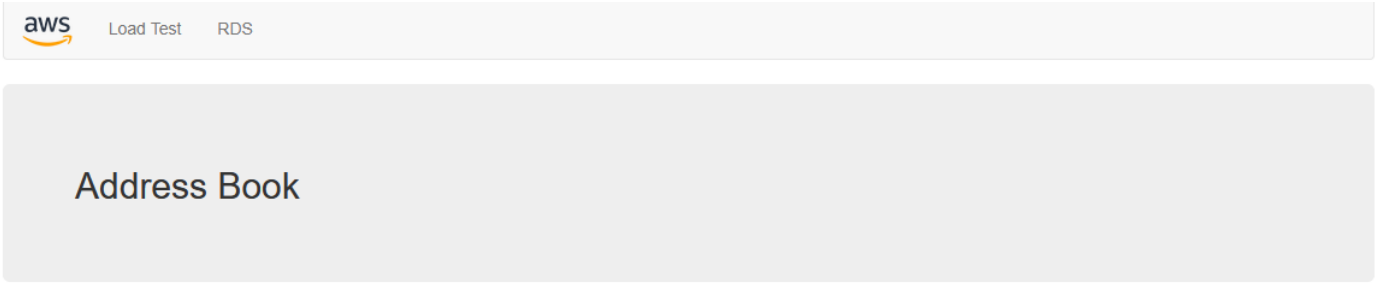


Figure 6 Webpage Showing Database Data

The current deployment demonstrates a limitation in availability in a scenario where the RDS instance or its associated resources were to fail, the database would be unavailable until AWS were able to recover the resources, which could take an undetermined amount of time resulting in downtime for the WinLocal web application. This would conflict with WinLocal's requirement of five nines availability. This design could be enhanced by maintaining a backup, replica database in a second availability zone with the same resources and an automatic failover.

Auto Scaling Configuration

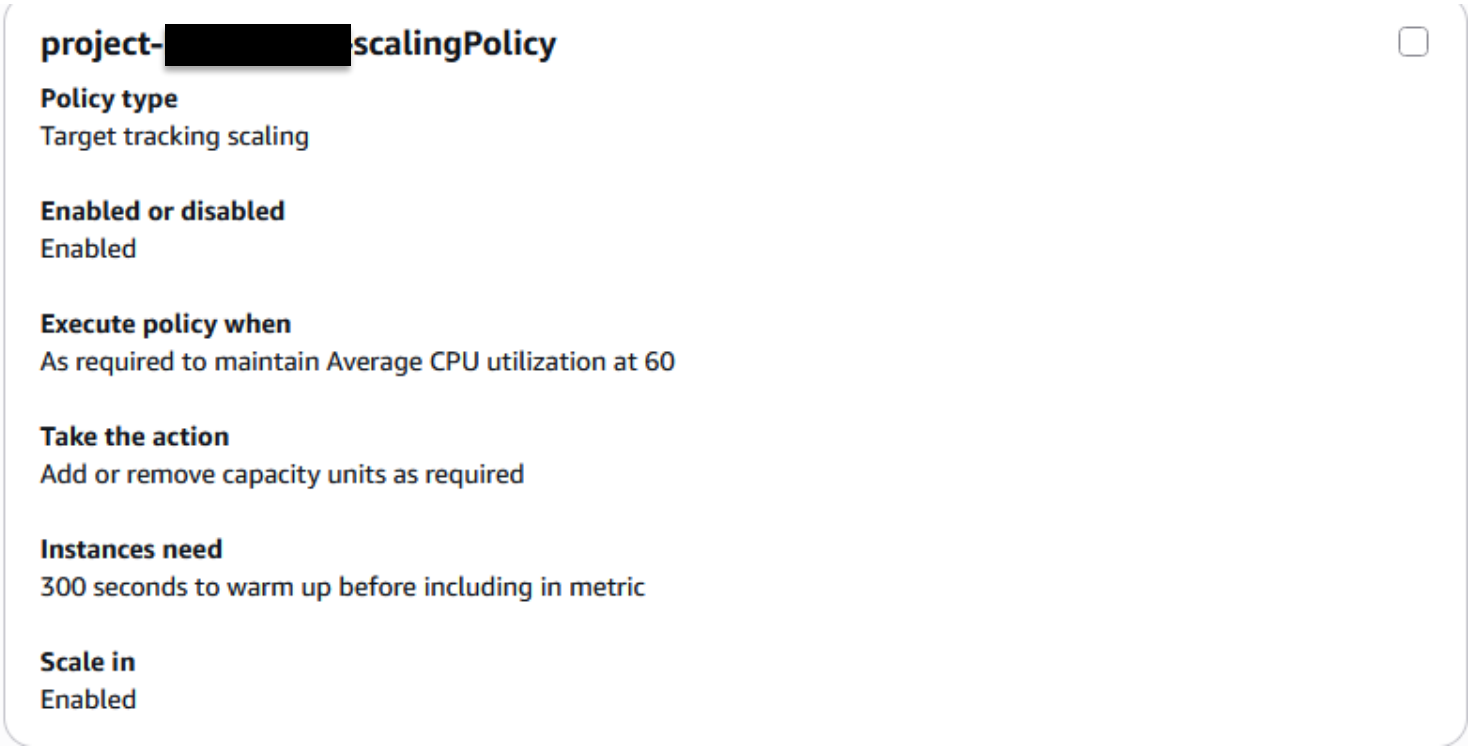


Figure 7 Auto Scaling Group Policy, Maintaining 60% Average CPU Usage

Figure 7 depicts an auto scaling group that was created using a launch template built from the previously discussed AMI, with a desired capacity of one instance, a minimum capacity of one instance and a maximum capacity of two instances, as required by WinLocal.

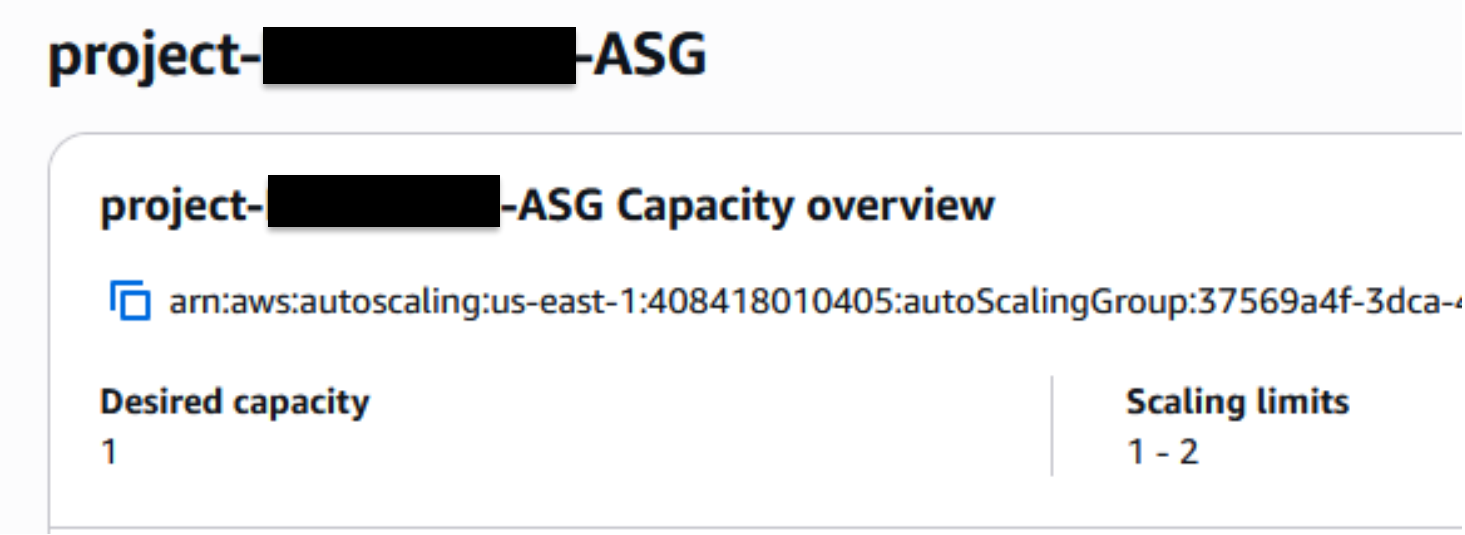


Figure 8 Auto Scaling Group Capacity Settings

A target tracking scaling policy was applied to the group to maintain an average CPU utilisation of 60% across all instances in the group. When CPU utilisation exceeded that limit, the auto scaling group triggers the policy to provision a second EC2 web instance automatically from the launch template and begin handling web traffic during periods of high load. When utilisation dropped below that threshold, the scaling group terminates the second instance, ensuring automatic scalability and sustainability with WinLocal only paying for the resources that are actively needed.

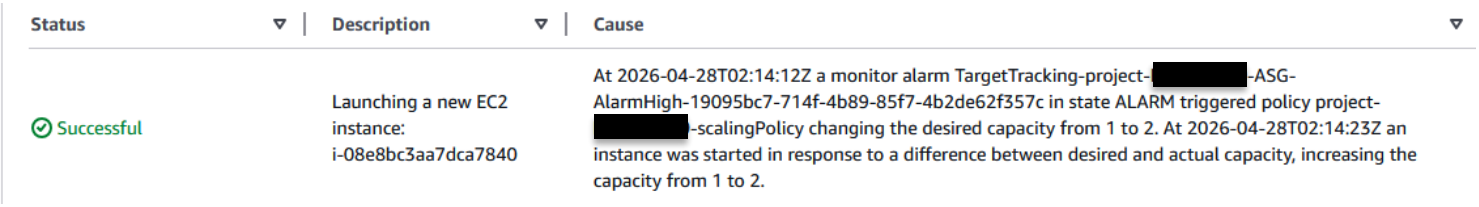


Figure 9 Scaling Policy Occurrence, Launching New Web Instances Automatically

Load Balancer Configuration

To forward internet traffic to the target group containing the auto scaling group web instances, an application load balancer was deployed across both public subnets in availability zone A and B, listening on ports 80 and 443 for HTTP(S) traffic. An application load balancer distributes incoming network traffic across multiple targets, performing health checks to route traffic only to healthy targets. This load balancer is a critical component of WinLocal's cloud solution as it replaces their outdated NetScaler load balancer, providing a highly available solution without the need for manual intervention, scaling automatically with increased traffic periods. The figures below demonstrate web traffic being routed to different instances in different availability zones during periods of increased traffic/CPU usage.

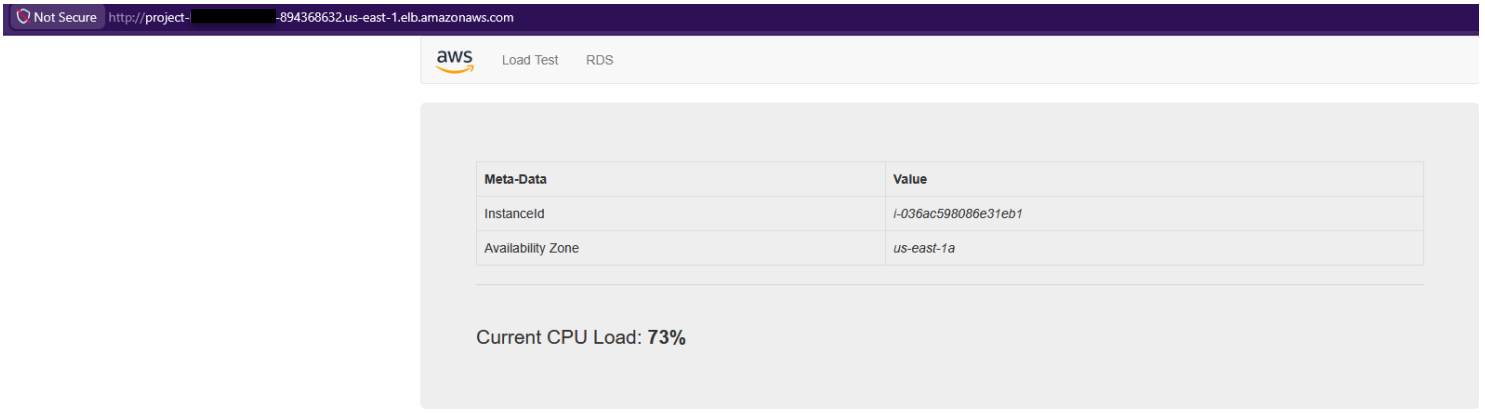


Figure 10 First Instance Being Served by Load Balancer in AZ-A

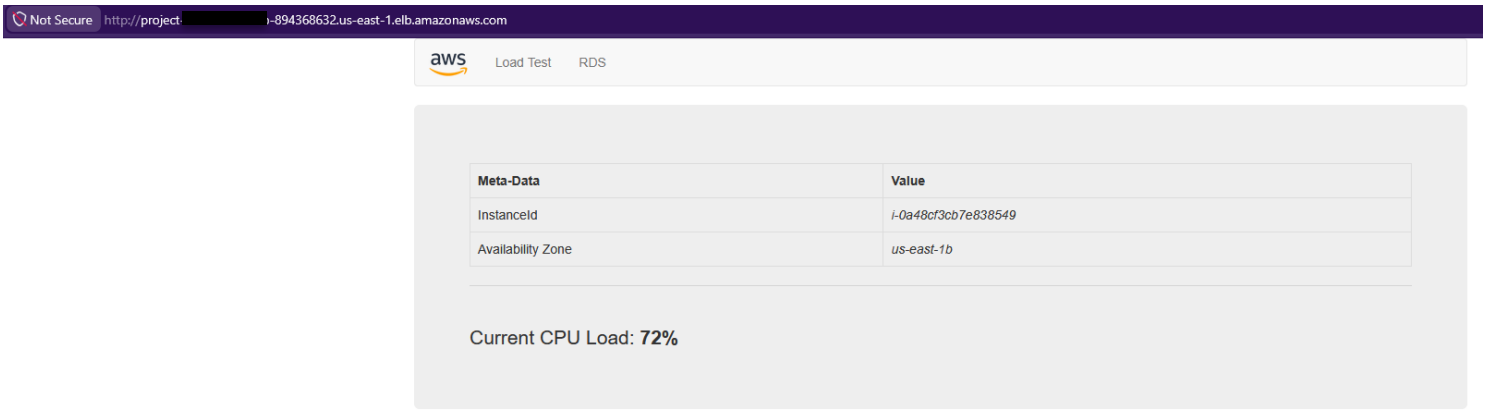


Figure 11 Second Instance Being Served by Load Balancer in AZ-B

Security Implementation

Security of the network and across the architecture is enforced through multiple security groups in each of the three layers. The load balancer security group permits inbound HTTP(S) traffic from any source for the public to reach the webpage. This is the only public facing entry point of the architecture.

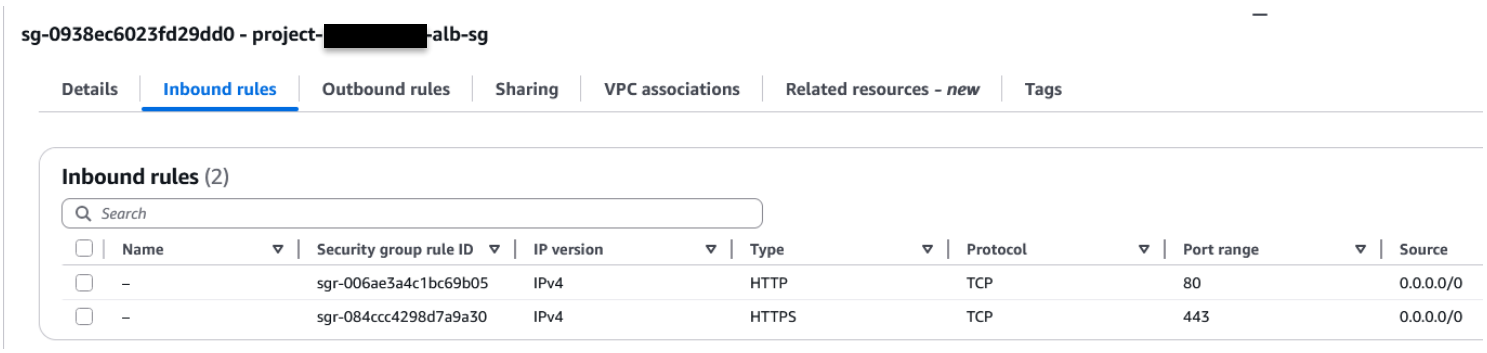


Figure 12 Security Group Allowing HTTP(S) Access to Load Balancer

The bastion host security group onbly permits SSH access on port 22 exclusively from a single trusted IP address.

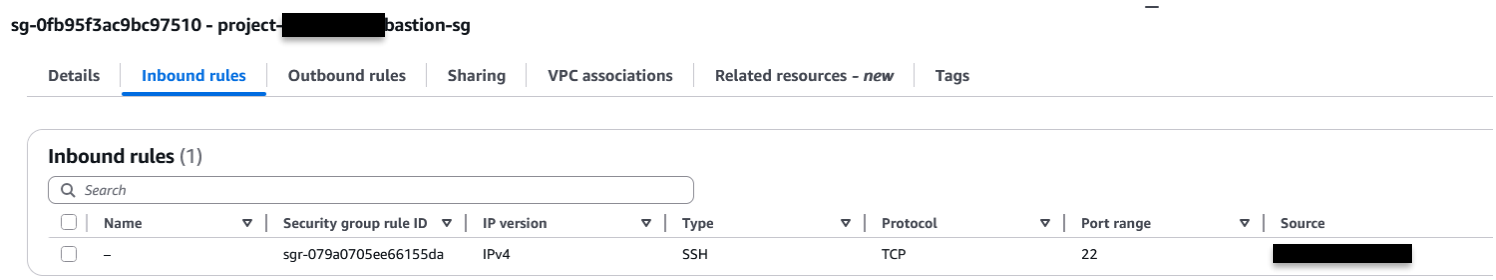


Figure 13 Security Group Allowing Restricted SSH Access to Bastion Host from Single IP

The web instance security group permits HTTP access on port 80, but only from the load balancer security group, while also allowing SSH on port 22 from the bastion host security group, enabling the ability to manage web instances from the bastion host instance. This keeps the web servers unreachable from the public internet.

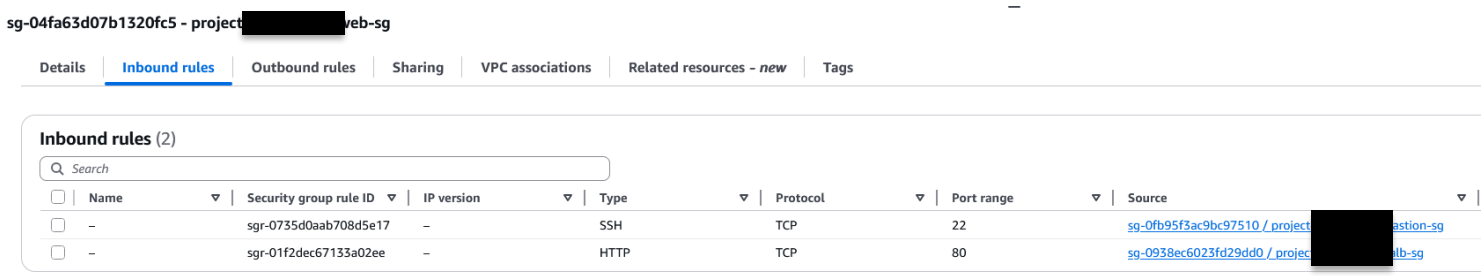


Figure 14 Security Group Allowing SSH Access to Web Instances from Bastion Host and HTTP Access from Load Balancer

Finally, the database security group permits MySQL traffic on port 3306 from the web instance security group exclusively, ensuring isolation and protection of customer data from the internet as it is only accessible through the application layer.

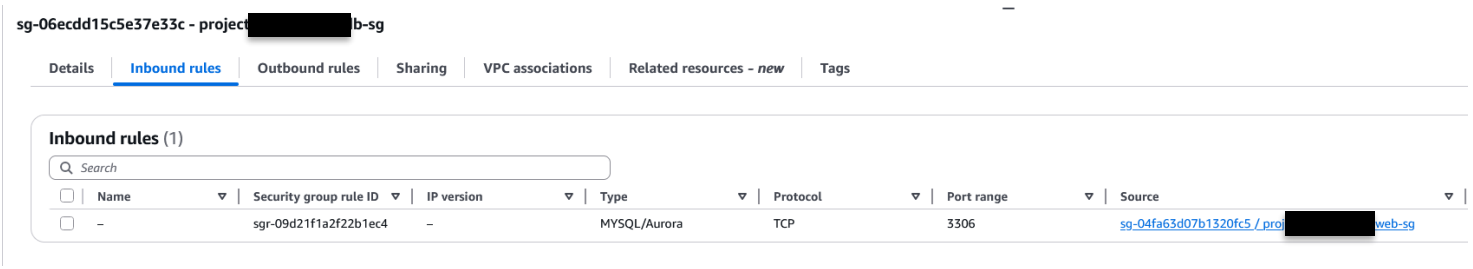


Figure 15 Security Group Allowing MySQL Access from Web Instances to Database

This multi-faceted security group approach means that in the event an attacker were to gain access to the load balancer or a web server, there would be additional security barriers before being able to access sensitive customer information. This approach directly addresses the vulnerability that allowed WinLocal to be targeted by a ransomware attack.

Static Backup Website

A static backup website was uploaded and hosted on an Amazon S3 bucket with a public read bucket policy applied, made accessible through a public URL. This backup site would be used in the event that the load balancer cannot serve a healthy instance to a web request and displays a simple service message informing users that the website is currently experiencing technical difficulties. This would be an important addition for WinLocal given their availability issues in their current environment.

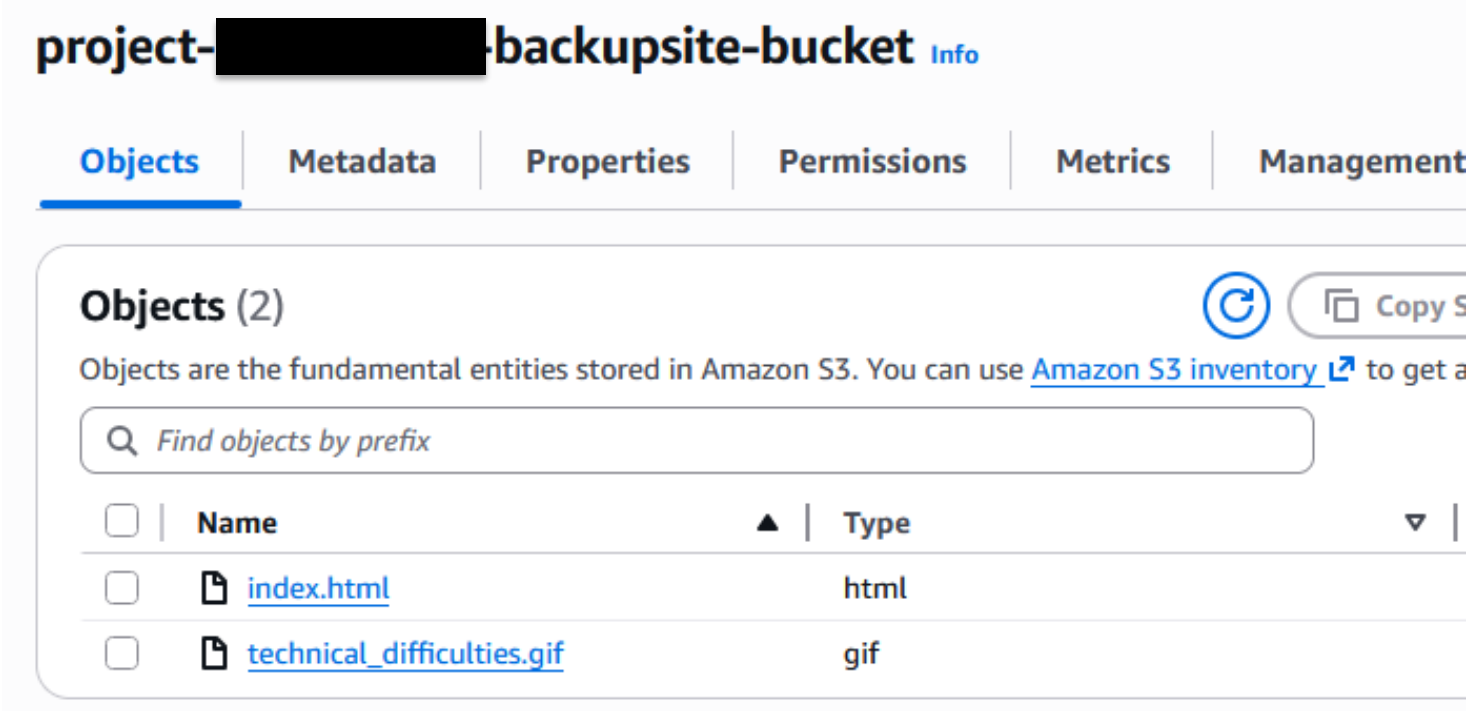


Figure 16 Static Site Hosted on Amazon S3

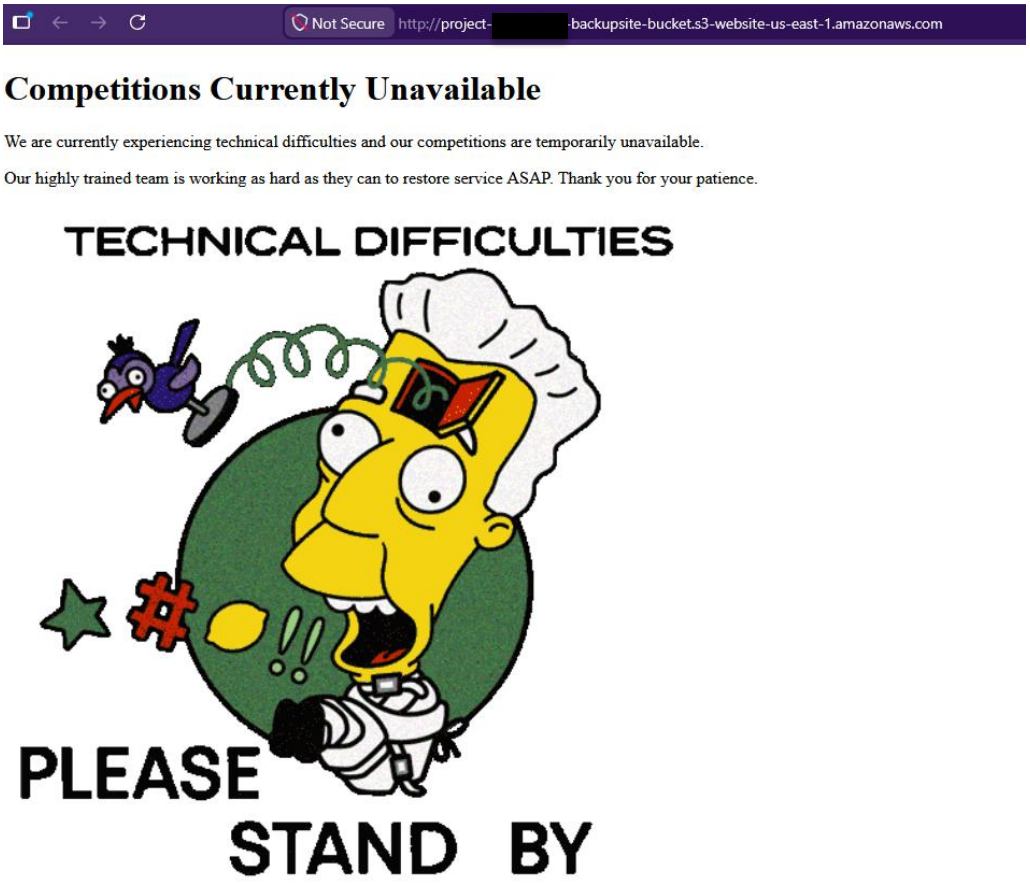


Figure 17 Static Site Publicly Accessible via URL

Additional AWS Services

Multi-AZ RDS Backup

The proposed architecture diagram includes a multi-az RDS backup replica database as an improvement over the current single-AZ deployment implemented in the proof of concept. In the current AWS environment, in a situation where an RDS instance were to fail, the database would become completely unavailable to WinLocal until AWS managed to complete a recovery procedure. This would contradict WinLocal's requirement for five nines of availability, a requirement that a single-AZ deployment would not be able to satisfy. Implementing a backup replica database in availability zone B, with an automatic failover solution, would keep availability high, require no manual intervention and would keep the database architecture more in line with the already implemented application layer architecture.

IAM Roles and Policies

IAM roles and policies represent a critical security enhancement for WinLocal's deployment. Their current IT team consists of one developer, two system administrators and plans to recruit three interns. This represents a mixed team of varying ability and seniority that would require varying levels of access. Without IAM roles, all team members could access any and all AWS resources regardless of their duties or responsibilities. IAM roles would enforce a policy of least privilege across the team, ensuring each member operates strictly within their limited permissions, and would provide an extra security control to prevent future attacks such as the ransomware attack that WinLocal has already been targeted by.

Sustainability

The proof of concept demonstrates several sustainable design choices. The auto scaling group for example ensures compute resources scale up during periods of high traffic, but also scale down during periods of low traffic to avoid wasting idle resources. The use of managed services (such as RDS and S3) means that the infrastructure on which those services are provided are shared and optimised by AWS rather than being dedicated solely to WinLocal's architecture. AWS is responsible for sustainability of the cloud while the customer is responsible for sustainability in the cloud, the proposed architecture makes efficient use of the resources that are being provisioned.